# CSGY6033 Homework - Sorting and Lower Bounds

Total: 100 points

## 1. [15 points]

**(a) [6 points]** Draw a comparison decision tree that sorts three distinct elements a, b, and c. Label every internal node with the comparison being performed and every leaf with the resulting sorted order. State the shortest and longest root-to-leaf path in your tree.

**(b) [4 points]** For n distinct elements, explain why the decision tree of any comparison-based sorting algorithm must have at least n! leaves.

**(c) [5 points]** Use the facts that a binary tree with L leaves has height at least log2 L and that n! >= (n/2)^(n/2) to show that any comparison-based sorting algorithm requires Omega(n log n) comparisons in the worst case. Briefly explain why this lower bound does not rule out linear-time Counting Sort, Radix Sort, or Bucket Sort under their required assumptions.

## 2. [15 points]

For each input below, choose the most appropriate algorithm from Quicksort, Counting Sort, Radix Sort, and Bucket Sort. Give the asymptotic running time and justify why the assumptions of the algorithm are satisfied. If more than one algorithm is reasonable, give the best asymptotic choice and explain.

**(a) [4 points]** n integers, each in the range 0 to 100.

**(b) [4 points]** n nonnegative integers, each at most n^2. You may choose the radix/base used by Radix Sort.

**(c) [4 points]** n real numbers drawn independently and uniformly from the interval [0, 1).

**(d) [3 points]** n arbitrary comparable objects with no known numeric range and no distributional assumptions. State both the expected and worst-case running time of your choice.

## 3. [20 points]

Consider the following standard version of Counting Sort for keys in the range 0, ..., k:

```
COUNTING-SORT(A, B, k)
1   let C[0 ... k] be a new array
2   for i = 0 to k
3     C[i] = 0
4   for j = 1 to A.length
5     C[A[j]] = C[A[j]] + 1
6   for i = 1 to k
7     C[i] = C[i] + C[i - 1]
8   for j = A.length downto 1
```

```
9       B[C[A[j]]] = A[j]
10      C[A[j]] = C[A[j]] - 1
```

**(a) [8 points]** Execute Counting Sort on A = [4, 2, 2, 8, 3, 3, 1] with k = 8. Show C after line 5, C after line 7, and the final output array B.

**(b) [6 points]** Counting Sort is a stable sorting algorithm. Explain why this is true, specifically referring to the direction in which the loop in line 8 processes the input array.

**(c) [6 points]** Suppose line 8 is changed to process j = 1 to A.length. Does the modified algorithm still sort correctly? Is it still stable? Justify your answer using two equal keys with different labels, such as 3a and 3b

**4. [20 points]**

Use the in-place PARTITION procedure from class. For this problem, always choose the last element of the current subarray as the pivot.

**(a) [8 points]** Execute QUICKSORT(A, 1, 9) on A = [9, 3, 7, 1, 8, 2, 5, 6, 4]. Show the array after every partition call and clearly indicate the pivot's final position and the parameters of the recursive calls.

**(b) [4 points]** Write and solve the recurrence for the best-case running time of Quicksort.

**(c) [4 points]** Write and solve the recurrence for the worst-case running time of Quicksort. Give an example of an input/pivot behavior that produces this case.

**(d) [4 points]** Randomized Quicksort chooses pivots uniformly at random. Explain, at a high level, why its expected running time is O(n log n) even though its worst-case running time is O(n^2). Your explanation should refer to balanced ('good') versus highly unbalanced ('bad') partitions.

**5. [15 points]**

**(a) [8 points]** Use least-significant-digit (LSD) Radix Sort in base 10 to sort the following values. Treat 51 as 051 and show the array after each digit pass: [329, 457, 657, 839, 436, 720, 355, 51].

**(b) [3 points]** Explain why the sorting algorithm used on each digit must be stable. Give a brief example of what could go wrong if it is not stable.

**(c) [4 points]** If each key has at most d digits and each digit has r possible values, derive the running time O(d(n + r)). State conditions on d and r under which Radix Sort runs in O(n) time.

**6. [15 points]**

**(a) [5 points]** Suppose n real numbers are drawn independently and uniformly from the interval [-5, 5]. Describe a Bucket Sort using n equal-width buckets. Give the formula or

rule that maps a value x to its bucket, explain how each bucket is sorted, and explain how the final output is produced.

**(b) [5 points]** Show why the expected running time of the algorithm in part (a) is O(n). Your argument should discuss the expected number of elements per bucket.

**(c) [5 points]** What is the worst-case running time if Insertion Sort is used inside each bucket? Describe an input distribution that causes it. If each bucket is instead sorted with Merge Sort, what worst-case bound can you guarantee for the overall algorithm? Justify your answer.